

UNIVERSIDADE FEDERAL DE GOIÁS
INSTITUTO DE INFORMÁTICA

GABRIEL FERREIRA SILVA

**Elicitação Automatizada de Requisitos
de Software a partir de Áudio de
Reuniões: uma Arquitetura Multiagente
Baseada em Inteligência Artificial
Generativa**

Goiânia
2026

UNIVERSIDADE FEDERAL DE GOIÁS
INSTITUTO DE INFORMÁTICA

**AUTORIZAÇÃO PARA PUBLICAÇÃO DE TRABALHO DE
CONCLUSÃO DE CURSO EM FORMATO ELETRÔNICO**

Na qualidade de titular dos direitos de autor, **AUTORIZO** o Instituto de Informática da Universidade Federal de Goiás – UFG a reproduzir, inclusive em outro formato ou mídia e através de armazenamento permanente ou temporário, bem como a publicar na rede mundial de computadores (*Internet*) e na biblioteca virtual da UFG, entendendo-se os termos “reproduzir” e “publicar” conforme definições dos incisos VI e I, respectivamente, do artigo 5º da Lei nº 9610/98 de 10/02/1998, a obra abaixo especificada, sem que me seja devido pagamento a título de direitos autorais, desde que a reprodução e/ou publicação tenham a finalidade exclusiva de uso por quem a consulta, e a título de divulgação da produção acadêmica gerada pela Universidade, a partir desta data.

Título: Elicitação Automatizada de Requisitos de Software a partir de Áudio de Reuniões: uma Arquitetura Multiagente Baseada em Inteligência Artificial Generativa

Autor(a): Gabriel Ferreira Silva

Goiânia, 1 de Dezembro de 2026.

Gabriel Ferreira Silva – Autor

Jacson Rodrigues Barbosa – Orientador

GABRIEL FERREIRA SILVA

Elicitação Automatizada de Requisitos de Software a partir de Áudio de Reuniões: uma Arquitetura Multiagente Baseada em Inteligência Artificial Generativa

Trabalho de Conclusão apresentado à Coordenação do Curso de Computação do Instituto de Informática da Universidade Federal de Goiás, como requisito parcial para obtenção do título de Bacharel em Computação.

Área de concentração: Otimização.

Orientador: Prof. Jacson Rodrigues Barbosa

Goiânia
2026

GABRIEL FERREIRA SILVA

Elicitação Automatizada de Requisitos de Software a partir de Áudio de Reuniões: uma Arquitetura Multiagente Baseada em Inteligência Artificial Generativa

Trabalho de Conclusão apresentado à Coordenação do Curso de Computação do Instituto de Informática da Universidade Federal de Goiás como requisito parcial para obtenção do título de Bacharel em Computação, aprovada em 1 de Dezembro de 2026, pela Banca Examinadora constituída pelos professores:

Prof. Jacson Rodrigues Barbosa

Instituto de Informática – UFG

Presidente da Banca

Prof. <Nome do membro da banca>

<Unidade acadêmica> – <Sigla da universidade>

Profa. <Nome do membro da banca>

<Unidade acadêmica> – <Sigla da universidade>

Todos os direitos reservados. É proibida a reprodução total ou parcial do trabalho sem autorização da universidade, do autor e do orientador(a).

Gabriel Ferreira Silva

<Texto com um perfil resumido do autor do trabalho. Por exemplo: (Graduou-se em Artes Cênicas na UFG - Universidade Federal de Goiás. Durante sua graduação, foi monitor no departamento de Filosofia da UFG e pesquisador do CNPq em um trabalho de iniciação científica no departamento de Biologia. Durante o Mestrado, na USP - Universidade de São Paulo, foi bolsista da FAPESP e desenvolveu um trabalho teórico na resolução do Problema das Torres de Hanói. Atualmente desenvolve soluções para problemas de balanceamento de ração para a pecuária de corte.)>

<Dedicatória do trabalho a alguma pessoa, entidade, etc.>

Agradecimentos

<Texto com agradecimentos àquelas pessoas/entidades que, na opinião do autor, deram alguma contribuição relevante para o desenvolvimento do trabalho.>

<Epígrafe é uma citação relacionada com o tópico do texto>

**<Nome do autor da citação>,
<Título da referência à qual a citação pertence>.**

Resumo

Ferreira Silva, Gabriel. **Elicitação Automatizada de Requisitos de Software a partir de Áudio de Reuniões: uma Arquitetura Multiagente Baseada em Inteligência Artificial Generativa**. Goiânia, 2026. 43p. Relatório de Graduação. Instituto de Informática, Universidade Federal de Goiás.

A eliciação de requisitos a partir de reuniões é uma atividade cara, manual e suscetível à perda de informação, na qual o profissional precisa, ao mesmo tempo, participar da discussão e registrá-la em artefatos formais. Este trabalho propõe um sistema multiagente, baseado em inteligência artificial generativa, que automatiza esse processo de ponta a ponta, transformando o áudio de uma reunião em documentação de requisitos estruturada. A arquitetura é composta por quatro agentes especializados, responsáveis, respectivamente, pela transcrição automática do áudio, pela identificação de personas e geração de histórias de usuário segundo o critério INVEST, pela geração do documento de requisitos em dois formatos selecionáveis (o padrão IEEE 830 e um formato específico de cliente) e pela síntese de um diagrama de domínio. Os agentes comunicam-se por meio de um *pipeline* sequencial baseado em arquivos, o que proporciona baixo acoplamento, isolamento de falhas e flexibilidade de ponto de entrada. O sistema foi validado em um caso real, por meio de uma parceria com um órgão público, tendo como referência um documento de requisitos oficial produzido manualmente. A avaliação, de caráter quantitativo, abrange a acurácia da transcrição, a qualidade da extração de informação, a qualidade percebida do documento por profissionais e a propagação de erro ao longo do *pipeline*.

Palavras-chave

Engenharia de Requisitos; Sistemas Multiagente; Inteligência Artificial Generativa; Histórias de Usuário; Reconhecimento Automático de Fala

Abstract

Ferreira Silva, Gabriel. **Automated Software Requirements Elicitation from Meeting Audio: a Multi-Agent Architecture Based on Generative Artificial Intelligence**. Goiânia, 2026. 43p. Relatório de Graduação. Instituto de Informática, Universidade Federal de Goiás.

Requirements elicitation from meetings is an expensive, manual, and error-prone activity, in which practitioners must simultaneously participate in the discussion and record it as formal artifacts. This work proposes a multi-agent system, based on generative artificial intelligence, that automates this process end to end, transforming meeting audio into structured requirements documentation. The architecture comprises four specialized agents responsible, respectively, for the automatic transcription of the audio, the identification of personas and the generation of user stories following the INVEST criteria, the generation of the requirements document in two selectable formats (the IEEE 830 standard and a client-specific format), and the synthesis of a domain diagram. The agents communicate through a file-based sequential pipeline, which provides loose coupling, fault isolation, and entry-point flexibility. The system was validated in a real-world setting, through a partnership with a public-sector agency, using a manually produced official requirements document as ground truth. The quantitative evaluation covers transcription accuracy, information extraction quality, the document quality as perceived by practitioners, and error propagation across the pipeline.

Keywords

Requirements Engineering; Multi-Agent Systems; Generative Artificial Intelligence; User Stories; Automatic Speech Recognition

Sumário

Lista de Figuras	11
1 Introdução	12
1.1 Objetivos	13
1.2 Questões de Pesquisa	14
1.3 Contribuições	14
1.4 Organização do Trabalho	15
2 Fundamentação Teórica	16
2.1 Engenharia de Requisitos e Elicitação	16
2.2 Histórias de Usuário e o Framework INVEST	17
2.3 Especificação de Requisitos de Software (IEEE 830)	17
2.4 Inteligência Artificial Generativa e Modelos de Linguagem	18
2.5 Sistemas Multiagente	19
2.6 Reconhecimento Automático de Fala	19
3 Trabalhos Relacionados	21
3.1 Modelos de Linguagem de Grande Escala na Engenharia de Requisitos	21
3.2 Reconhecimento de Fala para Transcrição de Reuniões	22
3.3 Qualidade de Histórias de Usuário e o Framework INVEST	22
3.4 Sistemas Multiagente para Engenharia de Software	23
3.5 Lacunas Identificadas	23
4 Arquitetura e Sistema Proposto	25
4.1 Visão Geral e Arquitetura do Sistema	25
4.2 Projeto dos Agentes	26
4.2.1 Agente 1 — Transcrição da Reunião	26
4.2.2 Agente 2 — Identificação de Personas e Extração de Histórias de Usuário	26
4.2.3 Agente 3 — Geração do Documento de Requisitos	27
4.2.4 Agente 4 — Síntese do Diagrama de Domínio	27
4.3 Estratégia de Seleção de Modelos	27
4.4 Estudo de Caso Real	28
5 Avaliação	29
5.1 Desenho da Avaliação	29
5.1.1 Conjuntos de Dados	29
5.1.2 Métricas e Protocolo por Questão de Pesquisa	29
5.2 Resultados e Discussão	31
5.2.1 QP1 — Acurácia da Transcrição	31

5.2.2	QP2 — Identificação de Personas e Histórias de Usuário	31
5.2.3	QP3 — Qualidade Percebida do Documento	31
5.2.4	QP4 — Cobertura frente ao Documento Oficial	31
5.2.5	QP5 — Propagação de Erro e Custo por Agente	31
5.3	Ameaças à Validade	31
6	Conclusão	32
6.1	Contribuições	32
6.2	Limitações	32
6.3	Trabalhos Futuros	32
6.4	Disponibilidade de Artefatos	32
	Referências Bibliográficas	33
A	Exemplo de um Apêndice	36
B	Exemplo de Outro Apêndice	40

Lista de Figuras

- 4.1 *Pipeline* de quatro agentes. Os agentes se comunicam por meio de um sistema de arquivos compartilhado, monitorado pela biblioteca *watchdog*. O Agente 4 é ignorado nas execuções no formato empresa.

26

Introdução

A Engenharia de Requisitos (ER) é uma fase fundamental do desenvolvimento de software, responsável por estabelecer a ponte entre as necessidades das partes interessadas e a implementação técnica do sistema. Na maioria das organizações, os requisitos são levantados em reuniões conduzidas por profissionais especializados — analistas de requisitos, *product owners* ou *scrum masters* —, cujo papel é capturar com precisão discussões complexas e em constante evolução e transformá-las em artefatos formais [3]. Esse fluxo de trabalho manual introduz riscos amplamente conhecidos: a perda de informação causada pelo intervalo de tempo entre a reunião e a etapa de documentação, a sobrecarga cognitiva do analista, que precisa simultaneamente participar da discussão e registrá-la, e a inconsistência de granularidade e de formato entre os artefatos resultantes. Como os documentos de requisitos servem de base para todas as decisões subsequentes do desenvolvimento, especificações incompletas ou de baixa qualidade traduzem-se diretamente em implementações desalinhadas, retrabalho e estouro de custos.

Ferramentas comerciais de inteligência de reuniões — como Fellow.ai [7], Jamie [12] e Otter.ai — automatizam a transcrição e a extração de itens de ação, mas produzem anotações de propósito geral, e não artefatos formais de requisitos de software. A tradução da discussão de uma reunião para uma Especificação de Requisitos de Software (do inglês *Software Requirements Specification* — SRS) estruturada permanece uma etapa manual e intensiva em conhecimento especializado. Na literatura acadêmica, os estudos que aplicam Modelos de Linguagem de Grande Escala (do inglês *Large Language Models* — LLMs) à ER tratam predominantemente de subtarefas isoladas: a geração de histórias de usuário a partir de requisitos textuais já existentes [6], a extração de modelos conceituais [23] ou a classificação de requisitos. Poucos trabalhos exploram *pipelines* completos, de ponta a ponta, que partam do áudio bruto de uma reunião e cheguem a uma documentação formalmente estruturada e específica para as partes interessadas [17].

Este trabalho aborda essa lacuna por meio de uma *arquitetura multiagente* na qual quatro agentes especializados colaboram para transformar o áudio de uma reunião em documentação de requisitos. Decompor o *pipeline* em agentes — em vez de utilizar um único *prompt* monolítico de LLM — oferece vantagens concretas de engenharia:

(i) *separação de responsabilidades*, permitindo que cada agente seja desenvolvido, testado e evoluído de forma independente; (ii) *flexibilidade de ponto de entrada*, permitindo que o usuário ingresse no *pipeline* em qualquer estágio (por exemplo, a partir de uma gravação ou de uma transcrição já existente); (iii) *seleção de modelo por agente*, ajustando a capacidade e o custo do LLM à complexidade de cada tarefa; e (iv) *propagação de erro observável*, tornando possível rastrear e medir como os erros de transcrição se propagam para os artefatos a jusante — uma preocupação de engenharia de primeira ordem em sistemas agênticos.

Experimentos iniciais com os Agentes 1 e 2, conduzidos em uma etapa anterior desta pesquisa, demonstraram 100% de acurácia na identificação de personas (28 de 28 personas ao longo de 7 reuniões do *corpus* AMI) e 83% de conformidade com o critério INVEST nas histórias de usuário geradas. Esta monografia estende aquela implementação à sua *forma completa*, acrescentando os Agentes 3 e 4 para a geração do documento de requisitos e a síntese do diagrama de domínio, introduzindo dois formatos de saída selecionáveis (a SRS no padrão IEEE 830 e um formato específico de cliente) e conduzindo uma validação no mundo real sobre um projeto de software real de um órgão público. Substitui-se, ainda, a avaliação qualitativa preliminar por um conjunto de métricas quantitativas que cobre a acurácia de transcrição, métricas de extração de informação, uma pesquisa de opinião com profissionais e a análise da propagação de erro ao longo do *pipeline*.

1.1 Objetivos

O objetivo geral deste trabalho é desenvolver e avaliar um sistema multiagente, baseado em inteligência artificial generativa, capaz de automatizar a elicitação de requisitos de software a partir do áudio de reuniões, produzindo documentos de requisitos estruturados.

Para atingir esse objetivo geral, foram definidos os seguintes objetivos específicos:

1. Implementar um *pipeline* de transcrição automática de áudio de reuniões em português, robusto a arquivos longos e a diferentes formatos de mídia.
2. Identificar automaticamente as personas (atores do sistema) presentes na reunião e gerar histórias de usuário segundo o critério INVEST.
3. Gerar documentos de requisitos estruturados em dois formatos distintos — o padrão IEEE 830 e um formato específico de cliente —, a partir das informações extraídas.
4. Validar o sistema em um caso real, por meio de uma parceria com a equipe de tecnologia de um órgão público, comparando o documento gerado com um documento oficial produzido manualmente (*gold standard*).

5. Avaliar quantitativamente o desempenho de cada agente e analisar empiricamente a propagação de erros entre os estágios do *pipeline*.

1.2 Questões de Pesquisa

A partir dos objetivos delineados, este trabalho busca responder às seguintes questões de pesquisa (QP):

- QP1.** Qual é a acurácia da transcrição automática (WER/CER) do Agente 1 para áudio de reuniões em português?
- QP2.** Com que precisão o Agente 2 identifica as personas das partes interessadas e gera histórias de usuário (Precisão/Revocação/F1 e conformidade com o INVEST)?
- QP3.** Qual é a qualidade percebida do documento de requisitos gerado, avaliada por profissionais da área por meio de uma pesquisa estruturada?
- QP4.** Quão completamente o documento gerado cobre uma especificação de requisitos oficial produzida manualmente (Precisão/Revocação/F1)?
- QP5.** Como os erros se propagam ao longo dos estágios do *pipeline* e como a seleção de modelo por agente afeta o compromisso entre custo e qualidade?

1.3 Contribuições

As principais contribuições deste trabalho são:

1. Um sistema multiagente completo e de código aberto, composto por quatro agentes, para a eliciação de requisitos de ponta a ponta a partir do áudio de reuniões, com suporte a dois formatos de documento (a SRS no padrão IEEE 830 e um formato específico de cliente).
2. Uma validação no mundo real sobre um projeto de software de um órgão público, utilizando como referência (*ground truth*) um documento de requisitos oficial produzido manualmente.
3. Uma avaliação quantitativa que abrange WER/CER (Agente 1), Precisão/Revocação/F1 com concordância entre avaliadores via *kappa* de Cohen (Agente 2) e uma pesquisa de opinião em escala Likert com profissionais, com avaliação de confiabilidade pelo alfa de Cronbach (Agente 3).
4. Uma análise empírica estruturada da propagação de erro entre agentes em um *pipeline* agêntico, acompanhada de medições de latência e de custo de API por estágio.

1.4 Organização do Trabalho

O restante deste trabalho está organizado da seguinte forma. O Capítulo 2 apresenta a fundamentação teórica, revisando os conceitos essenciais ao entendimento do trabalho: engenharia de requisitos e elicitação, histórias de usuário e o *framework* INVEST, a especificação IEEE 830, modelos de linguagem de grande escala, sistemas multiagente e reconhecimento automático de fala. O Capítulo 3 discute os trabalhos relacionados, posicionando esta pesquisa frente ao estado da arte e destacando as lacunas que a motivam. O Capítulo 4 descreve a arquitetura do sistema proposto, o projeto de cada um dos quatro agentes, a estratégia de seleção de modelos e o estudo de caso real. O Capítulo 5 apresenta o desenho da avaliação, os resultados obtidos e as ameaças à validade. Por fim, o Capítulo 6 apresenta as conclusões e os trabalhos futuros.

Fundamentação Teórica

Este capítulo apresenta os conceitos fundamentais para a compreensão deste trabalho. Inicialmente, discute-se a engenharia de requisitos e a atividade de elicitação (Seção 2.1). Em seguida, abordam-se as histórias de usuário e o *framework* INVEST (Seção 2.2) e a especificação de requisitos no padrão IEEE 830 (Seção 2.3). Por fim, apresentam-se os fundamentos de inteligência artificial generativa e modelos de linguagem (Seção 2.4), de sistemas multiagente (Seção 2.5) e de reconhecimento automático de fala (Seção 2.6).

2.1 Engenharia de Requisitos e Elicitação

A Engenharia de Requisitos (ER) é o ramo da engenharia de software dedicado a descobrir, documentar e manter o conjunto de requisitos de um sistema [20]. Um requisito descreve uma capacidade ou condição que o sistema deve atender para satisfazer as necessidades das partes interessadas (*stakeholders*). É prática consolidada distinguir os *requisitos funcionais*, que especificam o que o sistema deve fazer (serviços, comportamentos e transformações de dados), dos *requisitos não-funcionais*, que impõem restrições sobre como o sistema deve operar (desempenho, segurança, usabilidade, entre outros) [26].

O processo de ER costuma ser organizado em atividades de *elicitação*, *análise e negociação*, *especificação*, *validação* e *gerência* de requisitos [20]. A elicitação é a primeira e uma das mais críticas dessas atividades: é nela que o analista interage com as partes interessadas — por meio de entrevistas, *workshops* e, sobretudo, reuniões — para descobrir e compreender suas necessidades. A literatura é consistente ao apontar que erros introduzidos nesta fase são os mais caros de corrigir, pois se propagam para todas as etapas subsequentes do desenvolvimento; quanto mais tarde um defeito de requisito é detectado, maior o custo de sua correção [26].

Na prática das organizações, a elicitação é conduzida por profissionais especializados — analistas de requisitos, *product owners* ou *scrum masters* —, cuja tarefa é capturar discussões complexas e transformá-las em artefatos formais. Esse fluxo manual é trabalhoso e suscetível a falhas: há perda de informação entre a reunião e a documenta-

ção, sobrecarga cognitiva sobre o profissional que participa e registra simultaneamente, e inconsistência de granularidade entre os artefatos resultantes. A busca por métricas e por ferramentas que apoiem e automatizem essas atividades é, portanto, um tema de interesse contínuo na área [3].

2.2 Histórias de Usuário e o Framework INVEST

No desenvolvimento ágil de software, as *histórias de usuário* (*user stories*) são o artefato predominante para a representação de requisitos [5]. Uma história de usuário é uma descrição curta de uma funcionalidade, redigida da perspectiva de quem deseja utilizá-la, tipicamente no formato “*Como [papel], quero [objetivo] para [benefício]*” [15]. Mais do que um documento, a história de usuário é um convite à conversa: ela captura a intenção de forma concisa e adia o detalhamento para o momento da implementação.

Para avaliar se uma história de usuário está bem formulada, Wake [29] propôs o conjunto de heurísticas INVEST, acrônimo de seis critérios: *Independent* (independente de outras histórias), *Negotiable* (negociável, não um contrato fechado), *Valuable* (de valor para o usuário), *Estimable* (estimável em esforço), *Small* (pequena o suficiente para caber em uma iteração) e *Testable* (testável por meio de critérios de aceitação). O critério *Small* é, na prática, um dos mais difíceis de atender, pois histórias amplas demais comprometem a estimativa e o planejamento.

A avaliação sistemática da qualidade de histórias de usuário tem sido objeto de pesquisa. Lucassen et al. [15] formalizaram o *framework Quality User Story* (QUS), com 13 critérios de sintaxe, semântica e pragmática, e a ferramenta AQUISA para a detecção automática de defeitos. Xu et al. [31], por sua vez, combinaram o INVEST com um modelo de sete critérios para automatizar essa avaliação. Neste trabalho, o critério INVEST é adotado tanto como guia para a geração das histórias quanto como base para a sua avaliação.

2.3 Especificação de Requisitos de Software (IEEE 830)

Uma Especificação de Requisitos de Software (do inglês *Software Requirements Specification* — SRS) é o documento que registra, de forma estruturada, os requisitos de um sistema, servindo de acordo entre clientes e desenvolvedores e de base para o projeto, a implementação e os testes. O padrão IEEE 830 [11] estabelece uma prática recomendada para a elaboração desse documento, descrevendo as características desejáveis de uma boa SRS — correta, não ambígua, completa, consistente, verificável, modificável e rastreável — e propondo uma estrutura de referência.

Segundo o padrão, uma SRS organiza-se, em linhas gerais, em três partes: uma *introdução* (propósito, escopo, definições e visão geral), uma *descrição geral* (perspectiva e funções do produto, características dos usuários, restrições e dependências) e os *requisitos específicos*, nos quais se detalham os requisitos funcionais e não-funcionais, frequentemente apoiados por casos de uso. Os *casos de uso* descrevem a interação entre atores e o sistema para a realização de um objetivo, complementando a especificação textual dos requisitos. Neste trabalho, o padrão IEEE 830 é um dos dois formatos de saída suportados pelo sistema proposto (Capítulo 4).

2.4 Inteligência Artificial Generativa e Modelos de Linguagem

A inteligência artificial generativa designa a classe de técnicas capazes de produzir conteúdo novo — texto, imagem, áudio ou código — a partir de padrões aprendidos de grandes volumes de dados. No domínio do processamento de linguagem natural, esse avanço é impulsionado pelos *Modelos de Linguagem de Grande Escala* (do inglês *Large Language Models* — LLMs), redes neurais com bilhões de parâmetros treinadas para prever a continuação de uma sequência de texto e que, como consequência, adquirem capacidade de compreensão e geração de linguagem em domínios variados [10]. Esses modelos têm sido aplicados a uma ampla gama de tarefas de engenharia de software, da geração de código à melhoria automática da documentação de sistemas [27].

A maior parte dos LLMs modernos baseia-se na arquitetura *Transformer*, introduzida por Vaswani et al. [28], cujo mecanismo de *autoatenção* permite ponderar a relevância de cada elemento da sequência em relação aos demais, capturando dependências de longo alcance de forma paralelizável. A interação com esses modelos se dá pela *engenharia de prompts*, isto é, a formulação cuidadosa das instruções de entrada para orientar a saída do modelo — recurso central neste trabalho, em que cada agente é especializado por meio de *prompts* próprios.

Um padrão de uso particularmente relevante para a avaliação é o de *LLM como juiz* (*LLM-as-judge*), no qual um modelo de linguagem é empregado para avaliar artefatos segundo critérios definidos — por exemplo, julgar a conformidade de uma história de usuário com o INVEST. Por ser uma avaliação automática, esse padrão é escalável, mas requer validação contra avaliadores humanos para que se possa confiar em seus julgamentos, conforme adotado neste trabalho.

2.5 Sistemas Multiagente

Um *agente* é uma entidade computacional situada em um ambiente, capaz de perceber esse ambiente e de agir sobre ele de forma autônoma para atingir seus objetivos [30]. Um *sistema multiagente* (do inglês *Multi-Agent System* — MAS) é composto por múltiplos agentes que interagem entre si, por cooperação ou coordenação, para resolver problemas que estariam além da capacidade — ou da conveniência — de um único agente.

Com a popularização dos LLMs, arquiteturas multiagente passaram a ser aplicadas a tarefas de engenharia de software, atribuindo a agentes especializados papéis distintos que colaboram para produzir um resultado. Um levantamento recente de Cai et al. [4], que analisou 94 trabalhos sobre sistemas multiagente baseados em LLMs para engenharia de software, identificou a *Cooperação Baseada em Papéis (Role-Based Cooperation)* como o padrão de projeto mais adotado. A decomposição de um problema em agentes especializados — em contraste com um único *prompt* monolítico — traz vantagens de engenharia como a separação de responsabilidades, o isolamento de falhas, a possibilidade de testar e evoluir cada agente de forma independente e a seleção do modelo mais adequado (em capacidade e custo) para cada tarefa. Essas propriedades fundamentam a arquitetura adotada neste trabalho.

2.6 Reconhecimento Automático de Fala

O reconhecimento automático de fala (do inglês *Automatic Speech Recognition* — ASR) é a tarefa de converter um sinal de áudio com fala em texto. É a tecnologia habilitadora deste trabalho, pois transforma o áudio bruto de uma reunião na transcrição que alimenta os estágios seguintes. Radford et al. [22] introduziram o Whisper, um modelo treinado por supervisão fraca em larga escala que alcança transcrição multilíngue robusta sob condições acústicas adversas — ruído de fundo, sotaques e falantes não nativos —, características comuns em reuniões reais.

O ASR em reuniões de elicitación, contudo, enfrenta desafios específicos: vocabulário técnico, fala sobreposta e siglas próprias de cada projeto, que modelos de propósito geral podem transcrever incorretamente. A qualidade de uma transcrição é tipicamente medida pela *Taxa de Erro de Palavra* (do inglês *Word Error Rate* — WER) e pela *Taxa de Erro de Caractere* (*Character Error Rate* — CER). A WER é definida como

$$\text{WER} = \frac{S + D + I}{N},$$

em que S , D e I são, respectivamente, o número de substituições, exclusões e inserções

necessárias para transformar a transcrição produzida na transcrição de referência, e N é o número total de palavras na referência. A CER é calculada de forma análoga, no nível de caracteres. Essas métricas fundamentam a avaliação do primeiro agente do sistema (Capítulo 5).

Trabalhos Relacionados

Este capítulo revisa os trabalhos relacionados em quatro frentes: a aplicação de modelos de linguagem de grande escala à engenharia de requisitos, o reconhecimento de fala para a transcrição de reuniões, a avaliação da qualidade de histórias de usuário e os sistemas multiagente voltados à engenharia de software. Ao final, sintetizam-se as lacunas identificadas na literatura que motivam esta pesquisa.

3.1 Modelos de Linguagem de Grande Escala na Engenharia de Requisitos

A aplicação de modelos de linguagem de grande escala à engenharia de requisitos tem atraído interesse crescente [10]. Motger et al. [17] conduziram um estudo de mapeamento sistemático que identificou 62 conjuntos de dados publicamente disponíveis para pesquisa em ER baseada em LLMs, constatando que apenas 2 deles tratam de tarefas de elicitação e que 58 dos 62 são exclusivamente em inglês — uma escassez que motiva diretamente a validação em língua portuguesa apresentada neste trabalho.

No que diz respeito à geração de artefatos, Delgado et al. [6] compararam heurísticas baseadas em *N-gram* com o GPT-3 para a geração automática de histórias de usuário a partir de especificações textuais já existentes, reportando ROUGE-N = 0,46 e BERTScore = 0,69 para a abordagem baseada em LLM. Ren et al. [23] demonstraram, ainda, que a técnica de *Chain-of-Thought* permite que LLMs extraiam diagramas de classes conceituais a partir de histórias de usuário. Naimi et al. [18] utilizaram LLMs para gerar a descrição textual de casos de uso a partir de diagramas UML codificados como XML, reduzindo o tempo de documentação e melhorando a consistência. Em conjunto, esses trabalhos demonstram a viabilidade dos LLMs para a geração de artefatos de ER, mas compartilham um pré-requisito comum: a entrada textual estruturada. Nenhum deles trata da extração de requisitos a partir do áudio não estruturado de reuniões.

3.2 Reconhecimento de Fala para Transcrição de Reuniões

O reconhecimento automático de fala (do inglês *Automatic Speech Recognition* — ASR) é a tecnologia habilitadora da elicitación de requisitos baseada em áudio. Radford et al. [22] introduziram o Whisper, um modelo treinado por supervisão fraca em larga escala que demonstra transcrição multilíngue robusta sob diversas condições acústicas, incluindo falantes não nativos e ruído de fundo — características comuns em reuniões reais de partes interessadas. Kasakowskij e Haake [14] desenvolveram o T³ Talk2Text, um sistema de ASR em tempo quase real para reuniões virtuais em grupo, que captura transcrições de todo o grupo para a análise posterior de padrões de comunicação e colaboração.

Apesar desses avanços, o ASR em reuniões de elicitación de requisitos enfrenta desafios específicos do domínio: vocabulário técnico, fala sobreposta e siglas específicas de projeto que modelos de propósito geral podem transcrever incorretamente. Tais erros não permanecem isolados — eles se propagam para os agentes a jusante, um modo de falha que se analisa empiricamente na QP5.

3.3 Qualidade de Histórias de Usuário e o Framework INVEST

As histórias de usuário são o artefato de requisitos predominante no desenvolvimento ágil, seguindo o formato “*Como [papel], quero [objetivo] para [benefício]*” [15]. Wake [29] propôs as heurísticas INVEST — *Independent, Negotiable, Valuable, Estimable, Small e Testable* — como seis critérios para avaliar se uma história de usuário é acionável e bem delimitada.

Lucassen et al. [15] formalizaram o *framework* de *Quality User Story* (QUS), composto por 13 critérios que cobrem sintaxe, semântica e pragmática, e introduziram a AQUA, uma ferramenta baseada em processamento de linguagem natural que detecta defeitos de qualidade. Avaliada sobre 1.023 histórias de 18 organizações de software, a AQUA demonstra a viabilidade da avaliação automatizada de qualidade, mas requer histórias escritas manualmente como entrada. Xu et al. [31] combinaram o INVEST com um modelo de sete critérios para automatizar a avaliação de qualidade em *pipelines* ágeis. Este trabalho estende essa linha ao gerar histórias de usuário a partir do áudio de reuniões e ao avaliá-las frente ao INVEST por meio de um protocolo de LLM como juiz validado contra avaliadores humanos.

3.4 Sistemas Multiagente para Engenharia de Software

Arquiteturas multiagente emergiram como um paradigma produtivo para a automação de tarefas complexas de engenharia de software. O ChatDev [21] introduziu um *framework* baseado em diálogo no qual agentes LLM especializados colaboram por meio de conversas estruturadas ao longo das fases de projeto, codificação e teste, demonstrando que a comunicação linguística pode unificar agentes autônomos. O MetaGPT [9] codificou Procedimentos Operacionais Padronizados (SOPs) em sequências de *prompts*, atribuindo papéis de engenharia de software — gerente de produto, arquiteto, engenheiro e responsável por qualidade — a agentes distintos, alcançando resultados do estado da arte em *benchmarks* de geração de código. Luo et al. [16] aplicaram um *pipeline* agêntico à documentação, com três agentes especializados produzindo e mantendo a documentação de código em nível de repositório, superando as linhas de base produzidas por humanos. Há, ainda, esforços rumo a plataformas unificadas de desenvolvimento multiagente [24] e à automação de fluxos ágeis completos, da criação de histórias de usuário à geração e revisão de código [25]. Um levantamento recente de Cai et al. [4] analisou 94 artigos sobre sistemas multiagente baseados em LLMs para tarefas de engenharia de software, constatando que a *Cooperação Baseada em Papéis (Role-Based Cooperation)* é o padrão de projeto mais comumente adotado — o mesmo padrão que fundamenta o *pipeline* proposto neste trabalho.

O trabalho mais próximo do nosso é o MARE [13], um *framework* de colaboração multiagente para todo o processo de ER. O MARE decompõe a ER em quatro tarefas (elicitação, modelagem, verificação e especificação) distribuídas entre cinco agentes especializados, superando as linhas de base do estado da arte em até 15,4% em F1. Contudo, o MARE opera a partir de uma *ideia inicial (rough idea)* textual fornecida por um humano e simula a elicitação por meio de diálogo entre agentes; ele não processa o áudio de reuniões e, portanto, não pode ser implantado em um cenário pós-reunião totalmente automatizado. O sistema proposto neste trabalho aborda precisamente essa lacuna.

3.5 Lacunas Identificadas

A literatura revisada revela quatro lacunas que motivam este trabalho:

1. **Ausência de *pipelines* de áudio para SRS.** As ferramentas comerciais existentes [7, 12] produzem anotações de reunião de propósito geral, e não artefatos formais de ER. Os sistemas de ER baseados em LLMs, como o MARE [13], processam texto estruturado, e não o áudio conversacional bruto.
2. **Foco em subtarefas isoladas.** O trabalho anterior com LLMs em ER trata de tarefas individuais — geração de histórias de usuário [6], descrição de casos de

uso [18], modelagem conceitual [23] — sem integrar a transcrição, a identificação de personas e a geração de documentos em um único *pipeline*.

3. **Falta de validação no mundo real.** A maioria dos estudos utiliza conjuntos de dados sintéticos ou *corpora* controlados; poucos avaliam o resultado frente a um documento de requisitos oficial produzido manualmente em um projeto de software real.
4. **Propagação de erro não analisada.** Em *pipelines* agênticos, os erros de transcrição se propagam em cascata para os artefatos a jusante. Esse modo de falha é reconhecido, mas não é medido empiricamente em nenhum dos trabalhos relacionados.

Este trabalho aborda as quatro lacunas por meio de um *pipeline* de quatro agentes que transforma o áudio de reuniões em documentação de requisitos estruturada, validado sobre um projeto de software governamental real e com a medição quantitativa da propagação de erro entre agentes.

Arquitetura e Sistema Proposto

Este capítulo descreve a arquitetura do sistema proposto, o projeto de cada um dos quatro agentes especializados, a estratégia de seleção de modelos por agente e o estudo de caso real utilizado para a validação.

4.1 Visão Geral e Arquitetura do Sistema

O sistema proposto é composto por quatro agentes especializados conectados por meio de um *pipeline* sequencial baseado em arquivos, ilustrado na Figura 4.1. Cada agente monitora o diretório de saída de seu predecessor utilizando a biblioteca *watchdog*; ao detectar um novo arquivo, dispara o processamento automaticamente, sem intervenção manual. Esse projeto proporciona baixo acoplamento entre os estágios, isolamento de falhas e artefatos intermediários auditáveis. O usuário pode ingressar no *pipeline* em qualquer estágio — fornecendo o áudio bruto, uma transcrição já existente ou um arquivo JSON de análise anterior —, o que permite o reaproveitamento parcial de resultados entre execuções.

O *pipeline* suporta dois formatos de saída selecionáveis, escolhidos uma única vez por execução: **IEEE 830**, a estrutura padrão de Especificação de Requisitos de Software utilizada ao longo desta pesquisa; e **formato empresa**, um *layout* de documento específico de cliente, solicitado pelo parceiro industrial (Seção 4.4). Nas execuções no formato empresa, o Agente 4 é ignorado, pois esse formato de cliente não inclui diagramas de domínio. O formato afeta apenas o *template* do documento e alguns *prompts* dos agentes; todos os estágios anteriores (transcrição, identificação de personas e geração de histórias de usuário) são agnósticos ao formato.

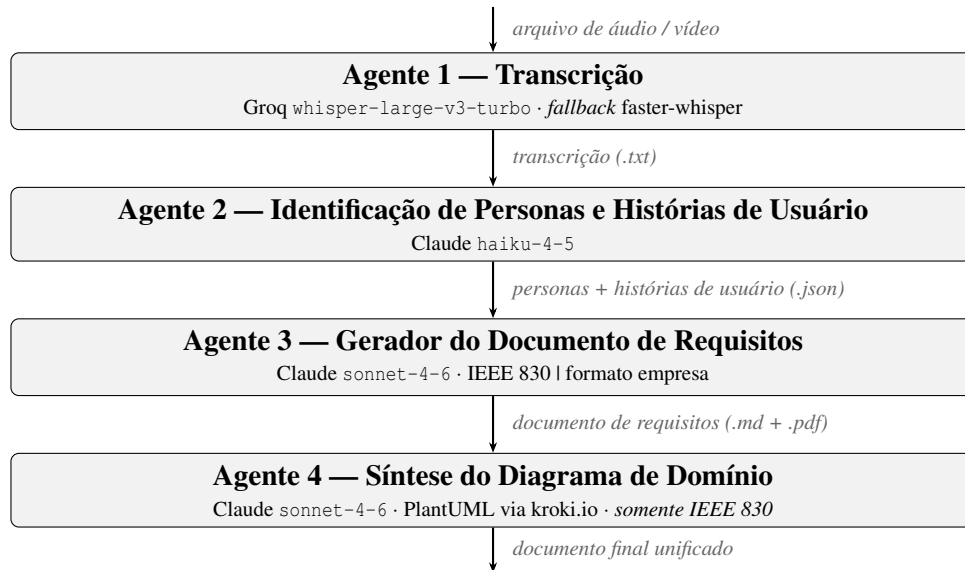


Figura 4.1: Pipeline de quatro agentes. Os agentes se comunicam por meio de um sistema de arquivos compartilhado, monitorado pela biblioteca watchdog. O Agente 4 é ignorado nas execuções no formato empresa.

4.2 Projeto dos Agentes

4.2.1 Agente 1 — Transcrição da Reunião

O Agente 1 aceita qualquer arquivo de áudio ou vídeo e produz uma transcrição em texto puro. O processamento começa com a normalização via **ffmpeg**: a entrada é convertida em um arquivo FLAC mono de 16 kHz, reduzindo o tamanho e preservando o sinal relevante para a transcrição. Se o arquivo normalizado exceder o limite de tamanho da API da Groq, ele é dividido em segmentos nas fronteiras de silêncio, cada um transcrito de forma independente e, ao final, mesclado em uma única saída.

A transcrição é realizada pela **Groq** executando o modelo `whisper-large-v3-turbo` [22, 8], escolhido por combinar alta acurácia multilíngue e baixa latência por meio de inferência acelerada por *hardware*. Caso a API da Groq esteja indisponível, o agente recorre automaticamente ao **faster-whisper**, executando o modelo `small` localmente [19], trocando acurácia por disponibilidade *offline*.

4.2.2 Agente 2 — Identificação de Personas e Extração de Histórias de Usuário

O Agente 2 lê a transcrição e executa duas chamadas sequenciais a um LLM, utilizando o **Claude haiku-4-5** [2]. A primeira chamada identifica as personas distintas das partes interessadas presentes na conversa, extraindo, para cada uma: nome ou identificador, papel, características definidoras e trechos representativos de fala. A segunda

chamada gera histórias de usuário específicas de cada persona, seguindo o critério INVEST [29] no formato “*Como [persona], quero [objetivo] para [benefício]*”. Decompor a identificação e a geração em duas chamadas reduz a interferência de contexto e melhora a estruturação da saída. Os resultados são salvos em um documento JSON tipado, que serve de contrato entre os Agentes 2 e 3.

4.2.3 Agente 3 — Geração do Documento de Requisitos

O Agente 3 lê o JSON produzido pelo Agente 2 e gera um documento de requisitos utilizando o **Claude sonnet-4-6** com *templates Jinja2*. No modo IEEE 830, a saída segue a estrutura padrão de SRS (introdução, descrição geral, partes interessadas, requisitos funcionais e tabela de histórias de usuário). No modo empresa, segue o *layout* do cliente (funcionalidades, requisitos funcionais detalhados com comentários, regras de negócio, casos de uso e requisitos não-funcionais). O documento é renderizado em *Markdown* e, em seguida, em PDF por meio do **WeasyPrint**, utilizando CSS *Paged Media* para gerar um sumário com numeração automática de páginas.

4.2.4 Agente 4 — Síntese do Diagrama de Domínio

O Agente 4 só é ativado nas execuções no formato IEEE 830. Ele lê tanto o documento do Agente 3 quanto o JSON original do Agente 2 e utiliza o **Claude sonnet-4-6** para extrair as entidades de domínio e seus relacionamentos. O diagrama de classes em PlantUML resultante é renderizado em PNG por meio da API pública **kroki.io** e incorporado a um documento de requisitos final unificado, que consolida todas as seções anteriores.

4.3 Estratégia de Seleção de Modelos

A seleção de modelo por agente reflete um compromisso explícito entre custo e qualidade. O Agente 2 realiza extração estruturada com um esquema de saída bem definido (JSON tipado); a tarefa é repetitiva e executada em todas as reuniões, o que torna o custo por chamada uma preocupação primária — daí o uso do modelo mais leve *haiku-4-5*. Os Agentes 3 e 4 realizam síntese aberta e geração de documentos, que exigem raciocínio mais forte e fluência de linguagem; para ambos, utiliza-se o *sonnet-4-6*. O Agente 1 emprega um modelo de ASR dedicado (Whisper) em vez de um LLM de propósito geral, pois o reconhecimento de fala é uma tarefa especializada na qual o Whisper alcança acurácia e velocidade substancialmente melhores.

4.4 Estudo de Caso Real

Para validar o sistema em um cenário autêntico, estabeleceu-se uma parceria com a equipe de tecnologia de um órgão público brasileiro. A equipe forneceu uma reunião gravada de partes interessadas e um documento de requisitos oficial, produzido manualmente para o mesmo sistema, que serve como referência (*ground truth*) para a avaliação (QP3–QP4).

A reunião envolveu **10 participantes** e teve duração de **1 hora, 37 minutos e 47 segundos**, conduzida inteiramente em português brasileiro. O tema foi a reformulação de um módulo de gestão de saldo orçamentário em um sistema governamental de planejamento financeiro. O arquivo de vídeo bruto (609 MB) foi processado de ponta a ponta pelo *pipeline*: normalização via ffmpeg, segmentação por silêncio, transcrição pela Groq, identificação de personas e geração do documento no formato empresa.

O formato empresa foi adotado a pedido do parceiro, uma vez que a organização não utiliza o IEEE 830 internamente. Esse requisito motivou a arquitetura de formato duplo: os estágios centrais de extração de dados são agnósticos ao formato, e o formato do documento é um parâmetro de saída configurável. O documento gerado e o documento oficial de referência são comparados quantitativamente no Capítulo 5.

Avaliação

Este capítulo apresenta a avaliação do sistema proposto. A Seção 5.1 descreve o desenho da avaliação — os conjuntos de dados, as métricas e o protocolo associado a cada questão de pesquisa. A Seção 5.2 reporta os resultados obtidos, e a Seção 5.3 discute as ameaças à validade do estudo.

5.1 Desenho da Avaliação

A avaliação foi estruturada em torno das cinco questões de pesquisa definidas no Capítulo 1, cada uma associada a um agente ou ao *pipeline* como um todo. Esta seção descreve, para cada questão, os dados utilizados, as métricas adotadas e o protocolo de medição.

5.1.1 Conjuntos de Dados

Três fontes de dados sustentam a avaliação:

- **Corpus AMI** — conjunto de reuniões anotadas [1], utilizado para a avaliação da identificação de personas e da geração de histórias de usuário (QP2). Sete reuniões foram processadas na etapa inicial desta pesquisa.
- **Corpus de áudio em português** — utilizado para a medição da acurácia de transcrição (QP1), que exige pares de áudio e transcrição de referência em português.
- **Estudo de caso real (parceiro público)** — a reunião gravada e o documento de requisitos oficial descritos na Seção 4.4, base das questões QP3 e QP4 e do estudo de propagação de erro (QP5).

5.1.2 Métricas e Protocolo por Questão de Pesquisa

QP1 — Acurácia da transcrição (Agente 1). Mede-se a Taxa de Erro de Palavra (WER) e a Taxa de Erro de Caractere (CER) da transcrição produzida frente à transcrição de referência, sobre o corpus de áudio em português. Reporta-se, ainda, o *Real-Time*

Factor (RTF), razão entre o tempo de processamento e a duração do áudio, e compara-se a transcrição via Groq com a execução local do *faster-whisper*.

QP2 — Identificação de personas e histórias de usuário (Agente 2). Avalia-se a identificação de personas por meio de Precisão, Revocação e F1, comparando as personas identificadas com os papéis de referência das reuniões. A qualidade das histórias de usuário é medida pela taxa de conformidade com os seis critérios INVEST, aplicada por um protocolo de LLM como juiz e validada em uma amostra por anotação manual. A concordância entre avaliadores é quantificada pelo *kappa* de Cohen.

QP3 — Qualidade percebida do documento (Agente 3). A qualidade do documento gerado é avaliada por profissionais por meio de uma pesquisa estruturada em escala Likert de cinco pontos, organizada em construtos (por exemplo, completude/acurácia, clareza/organização e utilidade/satisfação). Reportam-se a média e o desvio-padrão por construto, a consistência interna via alfa de Cronbach e a distribuição das respostas. Dada a natureza ordinal das respostas e o tamanho da amostra, adotam-se também medidas robustas (mediana) e testes não-paramétricos para comparações entre subgrupos.

QP4 — Cobertura frente ao documento oficial (Agente 3). Compara-se o documento gerado com o documento oficial produzido manualmente (*gold standard*), por meio de um protocolo de casamento item a item dos requisitos. A partir desse casamento, calculam-se a Precisão, a Revocação e o F1 da cobertura, separadamente para requisitos funcionais, regras de negócio e casos de uso. O casamento é conduzido por anotação manual, com validação por um segundo anotador (*kappa* de Cohen).

QP5 — Propagação de erro e custo por agente (*pipeline*). A propagação de erro é analisada como um estudo de caso qualitativo, rastreando como um erro de transcrição se manifesta nos artefatos a jusante até o documento final. Complementarmente, medem-se o tempo de processamento por estágio, o tempo total por reunião e o custo de API (estimado a partir do número de *tokens* consumidos por cada agente), oferecendo evidência sobre a viabilidade prática do sistema.

5.2 Resultados e Discussão

5.2.1 QP1 — Acurácia da Transcrição

5.2.2 QP2 — Identificação de Personas e Histórias de Usuário

5.2.3 QP3 — Qualidade Percebida do Documento

5.2.4 QP4 — Cobertura frente ao Documento Oficial

5.2.5 QP5 — Propagação de Erro e Custo por Agente

5.3 Ameaças à Validade

Discutem-se a seguir as principais ameaças à validade deste estudo, organizadas nas categorias de construto, interna, externa e de conclusão.

Validade de construto. Refere-se a se os instrumentos medem efetivamente o que se pretende medir. A qualidade percebida do documento (QP3) é um construto subjetivo; para mitigá-lo, a pesquisa foi organizada em construtos com múltiplos itens, e a consistência interna é verificada pelo alfa de Cronbach.

Validade interna. Diz respeito a fatores que poderiam enviesar os resultados. A pontuação de conformidade INVEST e o casamento de requisitos, quando realizados por um único anotador, estão sujeitos a viés de julgamento; mitiga-se essa ameaça com um segundo anotador e a medição da concordância via *kappa* de Cohen. Na pesquisa de opinião, o fato de parte dos respondentes ter participado da reunião pode influenciar a percepção, o que é tratado como uma variável de subgrupo na análise.

Validade externa. Refere-se à capacidade de generalização. A validação no mundo real apoia-se em um único caso real (o parceiro público), e o corpus AMI representa um cenário específico de reuniões; assim, os resultados não se generalizam automaticamente para todos os domínios. O tamanho reduzido da amostra da pesquisa também limita a generalização das conclusões da QP3.

Validade de conclusão. Relaciona-se à solidez das inferências estatísticas. Dado o tamanho reduzido da amostra e a natureza ordinal das escalas Likert, privilegiam-se testes não-paramétricos e a reportagem de medidas robustas (mediana e intervalo), além da média, evitando conclusões além do que os dados sustentam.

Conclusão

6.1 Contribuições

6.2 Limitações

6.3 Trabalhos Futuros

6.4 Disponibilidade de Artefatos

O código-fonte, os *scripts* de avaliação e os artefatos gerados nesta pesquisa estão publicamente disponíveis em <https://github.com/gabrielsfg/PFC1>, sob a licença MIT.

Referências Bibliográficas

- [1] AMI CONSORTIUM. **The AMI meeting corpus**. <https://huggingface.co/datasets/knkarthick/AMI>, 2024. Accessed: December 2024.
- [2] ANTHROPIC. **Claude model family**. <https://www.anthropic.com>, 2025. Accessed: June 2025.
- [3] BOKHARI, M. U.; SIDDIQUI, S. T. **Metrics for requirements engineering and automated requirements tools**. In: *Proceedings of the 5th National Conference on Computing for Nation Development (INDIACom)*, p. 10–11, New Delhi, India, 2011.
- [4] CAI, Y.; LI, R.; LIANG, P.; SHAHIN, M.; LI, Z. **Designing LLM-based multi-agent systems for software engineering tasks: Quality attributes, design patterns and rationale**. *ACM Transactions on Software Engineering and Methodology*, 2025. arXiv:2511.08475.
- [5] COHN, M. **User Stories Applied: For Agile Software Development**. Addison-Wesley, Boston, USA, 2004.
- [6] DELGADO, A.; OTHERS. **AI-driven user story generation**. In: *Proceedings of the 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, p. 1–12. IEEE, 2024.
- [7] FELLOW.AI. **Fellow: AI meeting assistant**. <https://fellow.ai>, 2025. Accessed: June 2025.
- [8] GROQ, INC.. **Groq API: Fast AI inference**. <https://groq.com>, 2025. Accessed: June 2025.
- [9] HONG, S.; ZHUGE, M.; CHEN, J.; ZHENG, X.; CHENG, Y.; ZHANG, C.; WANG, J.; WANG, Z.; YAU, S. K. S.; LIN, Z.; ZHOU, L.; RAN, C.; XIAO, L.; WU, C.; SCHMIDHUBER, J. **MetaGPT: Meta programming for a multi-agent collaborative framework**. In: *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*, 2024.

- [10] HORT, M.; VALLECILLOS-RUIZ, F.; MOONEN, L. **Large language models: A survey of surveys**. Preprint. <https://hal.science/hal-05341748>, 2025.
- [11] IEEE. **IEEE Recommended Practice for Software Requirements Specifications**. Institute of Electrical and Electronics Engineers, New York, USA, 1998.
- [12] JAMIE. **Jamie: AI note-taking assistant**. <https://www.meetjamie.ai>, 2025. Accessed: June 2025.
- [13] JIN, D.; JIN, Z.; CHEN, X.; WANG, C. **MARE: Multi-agents collaboration framework for requirements engineering**. *arXiv preprint arXiv:2405.03256*, 2024.
- [14] KASAKOWSKIJ, T.; HAAKE, J. M. **T³ Talk2Text: A model for near real-time voice transcription in virtual group meetings**. *Discover Education*, 4:146, 2025.
- [15] LUCASSEN, G.; DALPIAZ, F.; VAN DER WERF, J. M. E. M.; BRINKKEMPER, S. **Improving agile requirements: The quality user story framework and tool**. *Requirements Engineering*, 21(3):383–403, 2016.
- [16] LUO, Q.; YE, Y.; LIANG, S.; ZHANG, Z.; QIN, Y.; LU, Y.; WU, Y.; CONG, X.; LIN, Y.; ZHANG, Y.; CHE, X.; LIU, Z.; SUN, M. **RepoAgent: An LLM-powered open-source framework for repository-level code documentation generation**. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, p. 436–464. Association for Computational Linguistics, 2024.
- [17] MOTGER, Q.; CATOT, C.; FRANCH, X. **ShaRE your data! characterizing datasets for LLM-based requirements engineering**. In: *arXiv preprint arXiv:2510.18787*, 2025.
- [18] NAIMI, L.; BOUZIANE, E. M.; JAKIMI, A.; SAADANE, R.; CHEHRI, A. **Automating software documentation: Employing LLMs for precise use case description**. In: *Procedia Computer Science*, volume 246, p. 1346–1354. Elsevier, 2024.
- [19] OPENNMT. **CTranslate2: Fast inference engine for transformer models**. <https://github.com/OpenNMT/CTranslate2>, 2024.
- [20] POHL, K. **Requirements Engineering: Fundamentals, Principles, and Techniques**. Springer, Berlin, Heidelberg, 2010.
- [21] QIAN, C.; LIU, W.; LIU, H.; CHEN, N.; DANG, Y.; LI, J.; YANG, C.; CHEN, W.; SU, Y.; CONG, X.; XU, J.; LI, D.; LIU, Z.; SUN, M. **ChatDev: Communicative agents for software development**. In: *Proceedings of the 62nd Annual Meeting of the*

- Association for Computational Linguistics (ACL 2024)*. Association for Computational Linguistics, 2024.
- [22] RADFORD, A.; KIM, J. W.; XU, T.; OTHERS. **Robust speech recognition via large-scale weak supervision**. *arXiv preprint arXiv:2212.04356*, 2022.
- [23] REN, S.; NAKAGAWA, H.; TSUCHIYA, T. **LLM-based class diagram derivation from user stories with chain-of-thought promptings**. In: *Proceedings of the 2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*, p. 1376–1381. IEEE, 2024.
- [24] SAMI, M. A.; WASEEM, M.; RASHEED, Z.; SAARI, M.; SYSTÄ, K.; ABRAHAMSSON, P. **Experimenting with multi-agent software development: Towards a unified platform**, 2024.
- [25] SANWAL, M.; DEVA, I. **An autonomous multi-agent LLM framework for agile software development**. *International Journal of Trend in Scientific Research and Development (IJTSRD)*, 8(5):892–898, 2024. <https://www.ijtsrd.com/papers/ijtsrd70467.pdf>.
- [26] SOMMERVILLE, I. **Software Engineering**. Pearson, Boston, USA, 10 edition, 2015.
- [27] SU, Y.; WAN, C.; SETHI, U.; LU, S.; MUSUVATHI, M.; NATH, S. **HotGPT: How to make software documentation more useful with a large language model?** In: *Proceedings of the 19th Workshop on Hot Topics in Operating Systems (HotOS '23)*, p. 87–93, Providence, RI, USA, 2023. ACM.
- [28] VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, L.; POLOSUKHIN, I. **Attention is all you need**. In: *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [29] WAKE, B. **INVEST in good stories, and SMART tasks**. <https://xpl23.com/articles/invest-in-good-stories-and-smart-tasks/>, 2003. Accessed: June 2025.
- [30] WOOLDRIDGE, M. **An Introduction to MultiAgent Systems**. John Wiley & Sons, Chichester, UK, 2 edition, 2009.
- [31] XU, X.; DOU, Y.; QIAN, L.; ZHANG, Z.; MA, Y.; TAN, Y. **A requirement quality assessment method based on user stories**. *Electronics*, 12(9):2155, 2023.

Apêndicess são iniciados com o comando \apendices. Apêndicess são inicia-
dos com o comando \apendices. Apêndicess são iniciados com o comando \apendices.
Apêndicess são iniciados com o comando \apendices. Apêndicess são iniciados com o
comando \apendices. Apêndicess são iniciados com o comando \apendices. Apên-
dicess são iniciados com o comando \apendices. Apêndicess são iniciados com o co-
mando \apendices. Apêndicess são iniciados com o comando \apendices. Apêndi-
cess são iniciados com o comando \apendices. Apêndicess são iniciados com o co-
mando \apendices. Apêndicess são iniciados com o comando \apendices. Apêndicess
são iniciados com o comando \apendices. Apêndicess são iniciados com o comando
\apendices.

[illegible]

[illegible][illegible][illegible]